Notes on ray-tracing

J. Perdigon

March 13, 2025

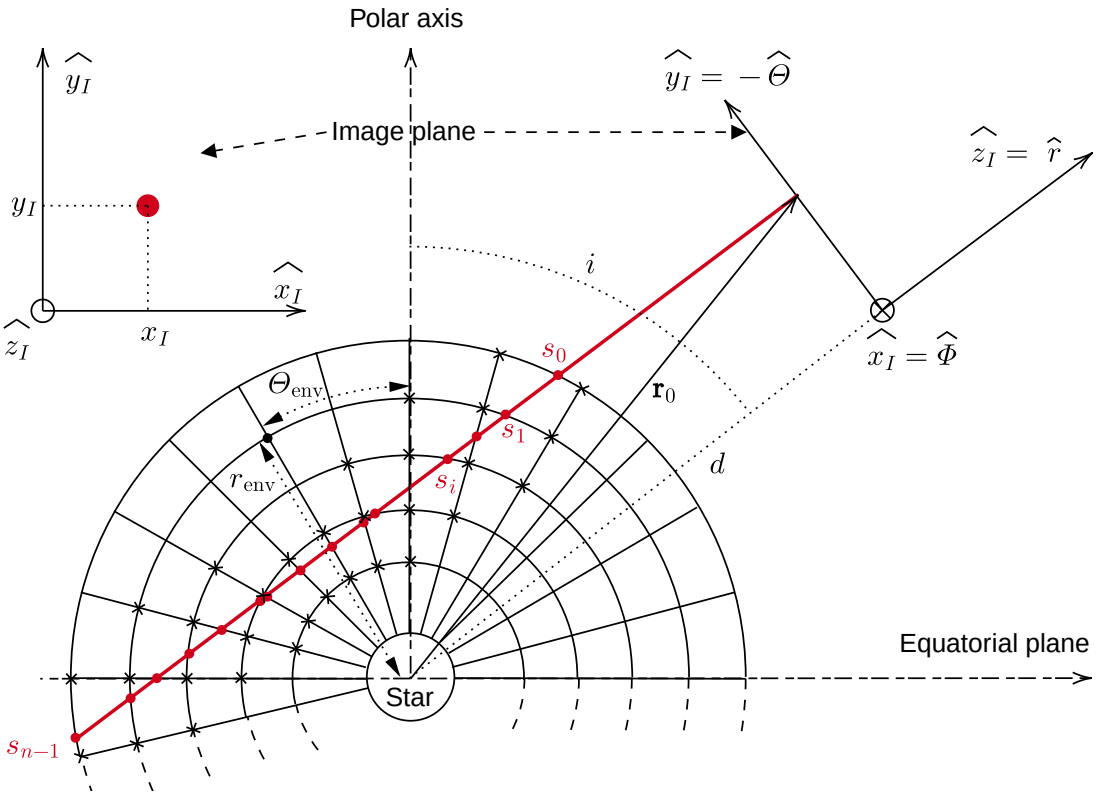

Figure 1: caption

0.1 Description of the ray tracing tool

In this section, we present the ray tracing tool[1] we developed in order to compute synthetic images from a given distribution of circumstellar material. The tool was first developed as a complementary module to the discontinuous Galerkin finite element method radiative transfer code from ref, for computing spectral energy distributions (SEDs hereafter). It was subsequently improved to include more features and developed as standalone package.

0.1.1 General overview

First, a given configuration of matter is sampled on the a regular grid around the star. This grid is defined with the spherical coordinate system. We further assume that the distribution of matter is symmetric around the polar axis and symmetric with respect to the equatorial plane. Each vertex of the grid is then only defined by two coordinates (r, Θ), with r being the radius and Θ the co-latitude angle with respect to the polar axis (see e.g. the black dot in Fig. 1).

The image plane is defined with respect to the spherical coordinates system of the grid. It makes an angle i with respect to the polar axis, and x_I and y_I axes coincide with the ϕ and $-\Theta$ axes of the spherical coordinates, respectively (see Fig. 1).

Each pixel constituting the image is intersected by a ray normal to the image plane (red line

[1]Publicly available at git link

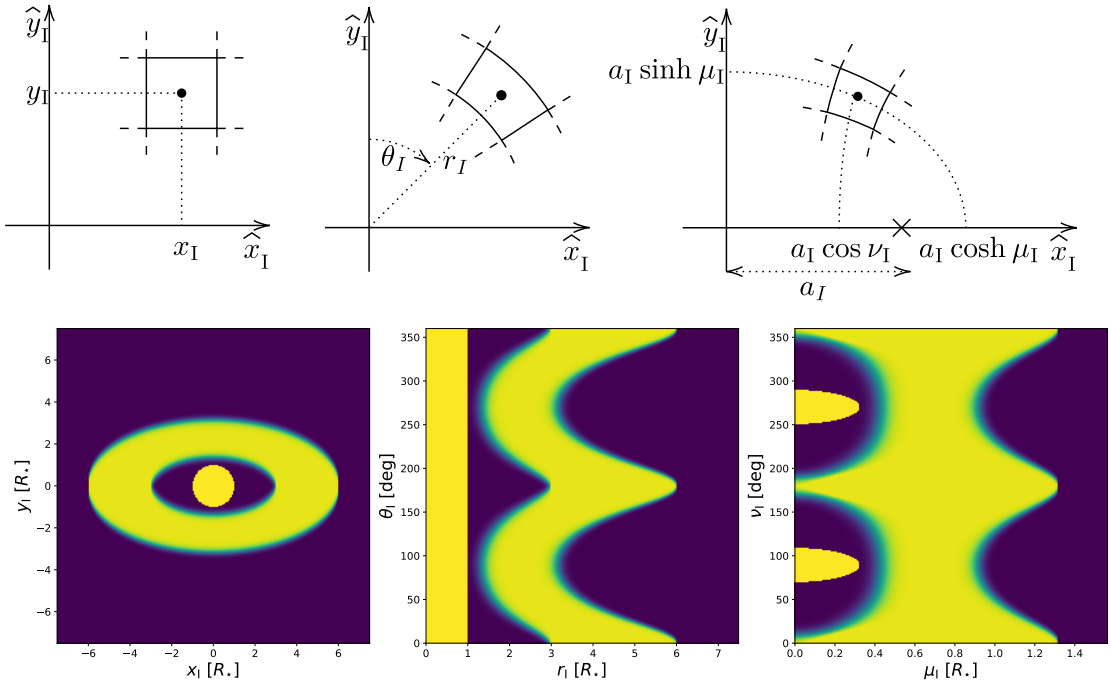

Figure 2: Example of an image of a dummy disc with $R_\mathrm{in} = 3\,R_\star$ and $R_\mathrm{out} = 5\,R_\star$, using three different geometries. The top panels display the coordinate system while the bottom ones show the corresponding image. *Left*: Cartesian geometry; the pixel surface ΔS is $\Delta x\,\Delta y$. *Middle*: Spherical geometry; $x_\mathrm{I} = r_\mathrm{I}\cos\theta_\mathrm{I}$, $y_\mathrm{I} = r_\mathrm{I}\sin\theta_\mathrm{I}$ and $\Delta S = r_\mathrm{I}\,\Delta r_\mathrm{I}\,\Delta\theta_\mathrm{I}$, with $r_\mathrm{I} \geq 0$ and $\theta_\mathrm{I} \in [0,\,2\pi]$. *Right*: Elliptic geometry; $x_\mathrm{I} = a_\mathrm{I}\cosh\mu_\mathrm{I}\cos\nu_\mathrm{I}$, $y_\mathrm{I} = a_\mathrm{I}\sinh\mu_\mathrm{I}\sin\nu_\mathrm{I}$ and $\Delta S = a_\mathrm{I}^2\left(\cosh^2\mu_\mathrm{I}\cos^2\nu_\mathrm{I} + \sinh^2\mu_\mathrm{I}\sin^2\nu_\mathrm{I}\right)\Delta\mu_\mathrm{I}\,\Delta\nu_\mathrm{I}$, with $\mu_\mathrm{I} \geq 0$ and $\nu_\mathrm{I} \in [0,\,2\pi]$. In this example, we set $a_\mathrm{I} = R_\mathrm{in}$.

Table 1: List of inputs and outputs of the ray tracing code

I/O	Description	Symbol	Python data type (shape)
Inputs	1st coordinates of pixels of the image	x_I	1D-ndarray $(N,)$
	2nd coordinates of pixels of the image	y_I	1D-ndarray $(N,)$
	Radial coordinates at the vertices of the circumstellar grid	r	1D-ndarray $(N_r,)$
	Angular coordinates at the vertices of the circumstellar grid	Θ	1D-ndarray $(N_\Theta,)$
	Extinction coefficient at the vertices of the circumstellar grid	κ^{ext}	3D-ndarray $(N_\lambda, N_r, N_\Theta)$
	Source function at the vertices of the circumstellar grid	S	3D-ndarray $(N_\lambda, N_r, N_\Theta)$
	Image plane inclination	i	float64 (1)
	Stellar radius	$R_\star$	float64 (1)
	Stellar intensity	$I_\nu^\star$	1D-ndarray $(N_\lambda,)$
Outputs	Specific intensity map	$I\left(\boldsymbol{r}_0,\,\hat{\boldsymbol{r}}\right)$	2D-ndarray $(N_\lambda,\,N)$

in Fig. 1), along which we want to estimate the specific intensity emitted by the circumstellar material. The intensity at the pixel centre $\boldsymbol{r}_0$, in the direction $\hat{\boldsymbol{r}}$, omitting the frequency dependence for clarity, can be expressed by the integral form of the radiative transfer equation ref,

$$I\left(\boldsymbol{r}_0,\, \hat{\boldsymbol{r}}\right) = \int_0^{\tau^{\max}} S\left(\tau\right) \exp\left(-\tau\right) d\tau. \tag{1}$$

where S is the source function and τ is the optical depth along the ray. The optical depth is defined as

$$\tau\left(s\right) = \int_{s_0}^{s} \kappa^{\text{ext}}\left(s'\right)\, ds', \tag{2}$$

with κ^{ext} the extinction coefficient and s, coordinate along the ray. Eqs. (1) and (2) are numerically estimated with the help of the values of S_ν and κ^{ext} at the intersections between the ray and the grid (red dots in Fig. 1). The values at the intersections are linearly interpolated from the closest neighbouring values of the grid (back crosses in Fig. 1). We give more technical details, in Appendix 0.1.2, on how the intersections and Eqs. (1) and (2) are computed.

The tool is written in the Python language and encapsulated in a class. The inputs parameters and outputs products are summarised in Table 1. In the following, we discuss about some points that might be relevant to the user when using the tool.

To compute an image, the user must input the x_{I} and y_{I} positions of each pixel centre. In this way, the tool can compute the intensity map from any given distribution of pixels. In Fig. 2, we show an example of an image of a dummy disc using 3 different geometries. While the Cartesian geometry has the advantage of allowing the use of the FFT algorithm to compute the physical observables presented in Sect. ref, it has the disadvantage of not featuring any pixel refinement. The spherical and elliptical geometries however naturally present some degree of pixel refinement, around the centre (x_{I}, $y_{\text{I}} = 0$, 0) and around the two focal points (x_{I}, $y_{\text{I}} = \pm a_{\text{I}}$, 0), respectively. This is a clear numerical advantage, because it means that objects with details around these refinements points will be resolved using a much lower number of pixels in comparison with the Cartesian geometry. The drawback of the the spherical and elliptic geometries is that one cannot use the FFT algorithm to compute the Fourier transform of the image. We discuss the latter point in more details in Sect. ref.

The user also need to provide the vertices (r, Θ) of circumstellar grid where the extinction coefficient κ^{ext} and the source function S_ν are defined. Because in general the values of these quantities span many order of magnitude across several order of magnitude in radius, it is usually necessary to logarithmic sample the radial coordinates of the vertices. For the polar angle Θ, the sampling should be done according the type of object considered. For instance, the user might want to increase the number of points towards the equatorial plane ($\Theta = \pi/2$) to correctly sample the density distribution of a circumstellar disc. If polar jets are present, the user might want to also refine the grid around the polar axis ($\Theta = 0$).

0.1.2 Evaluation of the pixel intensity

Let us denote by s the path coordinate along the ray, with an increasing value towards the star. The position of a given point on the ray is $\boldsymbol{r}\left(s\right) = \boldsymbol{r}_0 - s\,\hat{\boldsymbol{r}}$, with $\boldsymbol{r}_0$, the position of the intersection of the ray with the image plane, and $\hat{\boldsymbol{r}}$, the unit radial vector normal to the image plane.

In practice, we only know the values of S and κ^{ext} at the intersections of the ray with the grid (red dots in Fig. 1). We then need to assume a specific form for these quantities along each

portion between two consecutive intersections. The integral in Eq. (1) can be split as

$$I\left(\boldsymbol{r}_0,\,\hat{\boldsymbol{r}}\right)=\sum_{i=0}^{n-2}\int_{\tau(s_i)}^{\tau(s_{i+1})}S\left(\tau\right)\,\exp\left(-\tau\right)d\tau. \tag{3}$$

Performing the variable change $t=\tau-\tau(s_i)$,

$$I\left(\boldsymbol{r}_0,\,\hat{\boldsymbol{r}}\right)=\sum_{i=0}^{n-2}\exp\left(-\tau\left(s_i\right)\right)\int_{0}^{\Delta\tau_i}S\left(t\right)\,\exp\left(-t\right)dt, \tag{4}$$

with $\Delta\tau_i=\tau\left(s_{i+1}\right)-\tau\left(s_i\right)$. We assume S to be linear between two consecutive intersections

$$S\left(t\right)=\frac{S(s_{i+1})-S(s_i)}{\Delta\tau_i}\,t+S(s_i) \tag{5}$$

The integral in Eq. (4) is then

$$\begin{aligned}\int_{0}^{\Delta\tau_i}S\left(t\right)\,\exp\left(-t\right)dt=&\frac{1-\exp\left(-\Delta\tau_i\right)\left(1+\Delta\tau_i\right)}{\Delta\tau_i}\,S(s_{i+1})\\&-\frac{\Delta\tau_i-1+\exp\left(-\Delta\tau_i\right)}{\Delta\tau_i}\,S(s_i),\end{aligned} \tag{6}$$

which numerically diverges when $\Delta\tau\to 0$. To circumvent this problem, we replace Eq (6) by its 2nd order Taylor's expansion when $\Delta\tau_i<10^{-5}$,

$$\int_{0}^{\Delta\tau_i}S\left(t\right)\,\exp\left(-t\right)dt\underset{\Delta\tau_i<10^{-5}}{=}\frac{S(s_{i+1})+S(s_i)}{2}\,\Delta\tau_i. \tag{7}$$

The optical depth can be computed recursively since $\tau\left(s_i\right)=\tau\left(s_{i-1}\right)+\Delta\tau_i$, with the initial value $\tau\left(s_0\right)<10^{-5}$. The expression for $\Delta\tau_i$ is

$$\Delta\tau_i=\int_{0}^{\Delta s_i}\kappa^{\text{ext}}\left(s\right)\,ds, \tag{8}$$

with $\Delta s_i=s_{i+1}-s_i$. Again we assume κ^{ext} to be linear between two consecutive intersections, this yields the following expression for $\Delta\tau_i$,

$$\Delta\tau_i=\frac{\kappa^{\text{ext}}\left(s_{i+1}\right)+\kappa^{\text{ext}}\left(s_i\right)}{2}\,\Delta s_i \tag{9}$$

$$\boldsymbol{r}\left(s\right)=\boldsymbol{r}_0-s\,\hat{\boldsymbol{r}} \tag{10}$$

In the cartesian coordinate system, the position vector of a point along a given ray is

$$\boldsymbol{r}\left(s\right)=\begin{bmatrix}x(s)\\y(s)\\z(s)\end{bmatrix}=\begin{bmatrix}-x_{\text{I}}\\\sin i\left(d-s\right)-\cos i\,y_{\text{I}}\\\cos i\left(d-s\right)+\sin i\,y_{\text{I}}\end{bmatrix} \tag{11}$$

In the following, we will set the distance $d = 0$, for simplicity, since its value is irrelevant in the computations of intersections and in the ray integration. Once the s coordinate has been determined for a given intersection (see subsects ...), its radial coordinate can simply be determined using

$$r(s) = \sqrt{x^2 + y^2 + z^2} = \sqrt{s^2 + x_\mathrm{I}^2 + y_\mathrm{I}^2}. \tag{12}$$

The co-latitude angle Θ can subsequently be determined using

$$\Theta(s) = \arcsin\left(\sqrt{1 - \left(\frac{z(s)}{r(s)}\right)^2}\right) \tag{13}$$

In the following, we determine the s coordinate for the intersections with different shapes that constitute the spherical coordinate grid.

The equation of a sphere of radius r is, in Cartesian coordinates system,

$$x^2 + y^2 + z^2 = r^2. \tag{14}$$

To obtain an expression for s, we insert Eq. (11) in Eq. (14). Since the sphere is intrinsically spherically symmetric, we can arbitrarily choose $i = 0$ in order to simplify the computations. We then obtain a quadratic equation for s,

$$s^2 + x_\mathrm{I}^2 + y_\mathrm{I}^2 - r^2 = 0, \tag{15}$$

which has the trivial solutions,

$$s_\pm = \pm\sqrt{r^2 - x_\mathrm{I}^2 - y_\mathrm{I}^2}, \tag{16}$$

for rays verifying $x_\mathrm{I}^2 + y_\mathrm{I}^2 < r^2$. Note that we assume that tangent rays do not intersect the sphere.

The equation for a cone of opening angle Θ is

$$x^2 + y^2 = \tan^2\Theta\, z^2 \tag{17}$$

Again, inserting Eq. (11) in Eq. (17), we obtain the following quadratic equation for s,

$$\begin{aligned} K\, s^2 &+ 2\,\cos i\,\sin i\,\left(1 + \tan^2\Theta\right)\, y_\mathrm{I}\, s \\ &+ \left(\cos^2 i - \tan^2\Theta\,\sin^2 i\right)\, y_\mathrm{I}^2 + x_\mathrm{I}^2 = 0, \end{aligned} \tag{18}$$

with $K = \sin^2 i - \tan^2\Theta\,\cos^2 i$. The solution[2] for s is

$$s_\pm = \frac{-\cos i\,\sin i\left(1 + \tan^2\Theta\right)\, y_\mathrm{I} \pm \sqrt{\tan^2\Theta\, y_\mathrm{I}^2 - K\, x_\mathrm{I}^2}}{K} \tag{19}$$

for rays verifying $\tan^2\Theta\, y_\mathrm{I}^2 - K\, x_\mathrm{I}^2 > 0$.

[2] Again, we assume that tangent rays do not intersect the cone.

0.1.3 Flux, visibilities and closure phases

The ray tracing tool, described in Sect. 0.1, provides the user with the intensity map of the object. Formally, a pair of two telescopes, labelled 1 and 2, would produce a fringe pattern on the CCD of the instrument described by the Fourier transform of this intensity distribution,

$$\begin{aligned}\hat{I}_{1,2} &= \hat{I}\left(\frac{u_{1,2}}{\lambda},\, \frac{v_{1,2}}{\lambda}\right) \\ &= \int_{-\pi/2}^{\pi/2}\int_{-\pi/2}^{\pi/2} I\,(\alpha,\,\beta)\,\exp\left(-2\,i\,\pi\,\left(\alpha\,\frac{u_{1,2}}{\lambda}+\beta\,\frac{v_{1,2}}{\lambda}\right)\right)d\alpha\,d\beta,\end{aligned} \tag{20}$$

where $\alpha = \arctan\,(x_{\rm I}/d)$, $\beta = \arctan\,(y_{\rm I}/d)$, with d, the distance to the object. $u_{1,2}/\lambda$ and $v_{1,2}/\lambda$ are the conjugate variables of α and β, respectively, with $u_{1,2}$ and $v_{1,2}$, the components of the baseline for the pair of telescopes. The complex visibility is defined as,

$$V_{1,2} = V\left(\frac{u_{1,2}}{\lambda},\, \frac{v_{1,2}}{\lambda}\right) = \frac{\hat{I}_{1,2}}{\mathcal{F}} \tag{21}$$

where $\mathcal{F} = \hat{I}\,(0,\,0)$ is the observed flux. The closure phase of a triplet of three telescopes labelled 1, 2 and 3 is,

$$\mathcal{O}_{1,2,3} = \arg\,(V_{1,2}\,V_{2,3}\,{V_{1,3}}^{*}), \tag{22}$$

where ${V_{1,3}}^{*}$ is the complex conjugate of $V_{1,3}$.

The computation of the physical observables, Eqs. (**??**) and (**??**), from the intensity distribution $I(\alpha,\,\beta)$ of the model requires the numerical estimation of the Fourier transform Eq. (**??**). The Fast Fourier transform algorithm (FFT) is one of the most popular technique to efficiently compute the Discrete Fourier transform (DFT) associated with Eq. (**??**). However, the FFT requires an uniform sampling of the intensity distribution $I(\alpha,\,\beta)$, which is generally not the case in most of the image geometries (see Fig. 2). For non-uniform geometries, we may rather directly use the following mid-point rule for numerically estimating the integral in Eq. (**??**),

$$\hat{I}_{1,2} \approx \sum_{j=0}^{N-1} I\,(\alpha_j,\,\beta_j)\,\exp\left(-2\,i\,\pi\,\left(\alpha_j\,\frac{u_{1,2}}{\lambda}+\beta_j\,\frac{v_{1,2}}{\lambda}\right)\right)\Delta S_j, \tag{23}$$

with ΔS_j the surface of the $j^{\rm th}$ pixel. We recall that the FFT algorithm requires $\mathcal{O}\,(N\,\log N)$ operations. On contrary, the expression in Eq. (**??**), typically needs $\mathcal{O}\,(N_{u,\,v}\,N)$ operations, with $N_{u,\,v}$ the number of points in the (uv) plane. Consequently, Eq. (**??**) is numerically advantageous if $N_{u,\,v} < \log N$. Additionally, note that when we use Eq. (**??**), we directly evaluate $\hat{I}_{1,2}$ at the desired baseline coordinate $(u_{1,2},\,v_{1,2})$. Consequently no further interpolation step is needed, as it is the case when using the FFT.